

WISEConv: Worklist-driven Masked Convolution for Onboard High-Speed Robotic Perception

Anonymous Authors

Abstract—Onboard robotic perception relies on 2D convolutional networks that must meet hard latency constraints under the limited compute of embedded platforms. Many high-speed vision systems exploit spatial sparsity by producing an active mask and updating only the affected convolution outputs. However, existing approaches face a fundamental trade-off: tile skipping preserves the dense pipeline’s throughput but computes many unneeded outputs, while gather-scatter computes only needed outputs but sacrifices throughput. Neither converts high sparsity into proportional latency reduction. We present WISEConv, a masked convolution operator that achieves both high useful-compute ratio and near-dense throughput by fusing position-level selectivity into the dense pipeline itself. WISEConv first scans the mask to build an ordered worklist of active positions, then drives the standard convolution datapath using only those positions. On Jetson Xavier NX and Orin, across three mask sources and three robotic perception tasks, WISEConv achieves near-linear latency scaling with the active ratio, outperforming state-of-the-art baselines while preserving task accuracy.

Index Terms—Computer architecture for robotic and automation, software, deep learning methods, sparse convolution, embedded perception.

I. INTRODUCTION

ONBOARD robots run convolutional perception in the control loop for navigation, manipulation, and closed-loop control [1]–[3]. On embedded platforms the available compute is far below server-class GPUs, yet perception latency is a hard real-time constraint because it sits directly on the control path. Dense convolution recomputes every spatial position, and on these platforms it often cannot reach the update frequency the loop demands. Many high-speed vision systems on UAVs, mobile robots, and embedded devices [4]–[7] therefore concentrate compute on active regions via *masked convolution*: the input stays a dense image-grid tensor, but only the outputs at positions marked active are updated.

The standard dense convolution pipeline partitions the output grid into *tiles* (contiguous spatial blocks) and for each tile reads the required input patch by coordinate, computes the convolution, and writes the result back. This coordinate-driven, tile-based execution achieves high throughput because tiles access spatially contiguous data with regular, predictable memory patterns. Existing masked-convolution systems modify or abandon this pipeline along two paths.

Tile skipping [4], [6], [8], [9] stays inside the dense pipeline: it tests each tile and skips it if every position within is inactive. Throughput is preserved for non-skipped tiles, but a single active position forces the entire tile to execute, so many computed outputs are unneeded. *Gather-scatter* [7], [10] abandons the pipeline entirely: it extracts active positions exactly, gathers

their input neighborhoods, computes, and scatters results back. Every computed output is needed, but the extraction, irregular memory access, and scatter writeback collapse throughput.

We characterize this trade-off along two axes: *throughput* (operations per second, bounded by the dense pipeline) and *useful-compute ratio* (the fraction of executed operations that produce needed outputs). Tile skipping preserves throughput but sacrifices useful-compute ratio; gather-scatter achieves high useful-compute ratio but sacrifices throughput. Neither delivers both, so upstream sparsity does not translate into proportional latency reduction. In our evaluation on Jetson Xavier NX and Orin, existing methods [4], [6], [7], [10], [11] reduce the active-output ratio to roughly XX–YY%; yet tile-skipping methods achieve only AA–BB% useful-compute ratio, while gather-scatter methods reach only CC% of tile-skipping throughput. Convolution latency is reduced by only DD–EE%.

Our key insight is that the position-level selectivity of gather-scatter can be fused into the dense pipeline’s coordinate-driven execution without the separate data-movement stages. The fusion is straightforward in principle: because each tile already performs a coordinate-driven read-compute-write per position, replacing the tile’s coordinate source, from a contiguous block to a set of active positions, gives position-level selectivity while keeping the per-position execution path identical. This achieves high useful-compute ratio directly, and because the per-position datapath remains that of the dense pipeline, it opens the potential for dense-level throughput.

Realizing this potential requires addressing the challenge of two practical overheads. First, *coordinate discovery*: the kernel must know which positions are active before computation begins, yet scanning for them inside the compute kernel would stall the main datapath. Second, *locality*: if active positions are processed in arbitrary (instead of contiguous) order, memory access loses the spatial regularity that gives the dense pipeline its speed; a global sort would restore order but is itself expensive.

We present WISEConv (**W**orklist-driven **maSkEd** Convolution), a dense-grid masked convolution operator that resolves both overheads through a two-stage design. A lightweight construction stage scans the active mask and emits a *worklist*: the ordered coordinates of active output positions. The subsequent compute stage drives the dense pipeline’s coordinate-driven execution with the worklist as its coordinate source, computing only the active outputs. Both overheads are addressed by the construction stage’s tiled scan: it operates entirely in 2D mask space (no feature data touched), so coordinate discovery costs only XX% of the compute-stage time; and it compacts active

positions from each tile into contiguous worklist segments, so the compute stage accesses spatially nearby input data when processing consecutive worklist entries, preserving locality comparable to the dense traversal without any global sort. With construction overhead minimized and only active outputs computed, WISEConv achieves near-linear latency scaling: an active-output ratio of $x\%$ yields convolution latency close to $x\%$ of the dense baseline.

This paper makes the following contributions:

- We identify the useful-compute ratio vs. throughput trade-off as the fundamental bottleneck preventing masked convolution from delivering proportional speedup on embedded platforms [4], [6]–[10].
- We present WISEConv, a worklist-driven masked convolution operator that fuses position-level selectivity into the dense pipeline’s coordinate-driven execution, achieving both high useful-compute ratio and near-dense throughput.
- On Jetson Xavier NX and Orin, WISEConv achieves near-linear latency scaling with active-output ratio across three mask sources and three robotic perception tasks, outperforming dense cuDNN, tile skipping, and gather-scatter while preserving task accuracy.

II. BACKGROUND AND MOTIVATION

A. 2D Dense Convolution on Embedded Platforms

Convolutional neural networks (CNNs) are the computational backbone of onboard robotic visual perception. CNN-based depth estimation and optical flow [1], [2], [12] feed obstacle avoidance and path planning; object detection [13] and semantic segmentation [14] provide scene understanding for navigation; and pose estimation [3], [15] enables closed-loop manipulation. While vision transformers [16] have achieved strong accuracy on these tasks, their quadratic attention cost makes them impractical under the tight latency and power budgets of embedded platforms; CNN-based models remain the dominant deployed choice for real-time onboard visual inference. Across these models, convolutional layers operating on image-grid feature maps dominate the inference workload and sit directly on the control-critical path.

On embedded platforms (Jetson Xavier NX, Orin, edge TPUs), power and thermal budgets limit available compute to a fraction of server-class GPUs, yet the closed control loop imposes a hard latency ceiling: perception must complete within the sensor frame interval (typically tens of milliseconds) to feed downstream planning and control in time.

A 2D convolution layer transforms an input feature map $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$ into an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ using a weight tensor $\mathbf{W} \in \mathbb{R}^{C_{out} \times C_{in} \times k \times k}$, where k is the kernel size and $k \times k$ is the receptive field size. The computation is defined per position on the output grid $\mathcal{G}_{out} = \{1, \dots, H_{out}\} \times \{1, \dots, W_{out}\}$. For each $p \in \mathcal{G}_{out}$, let $\mathcal{N}(p) \subset \mathcal{G}_{in}$ denote the set of k^2 input positions in the receptive field of p . The layer computes:

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{W}_i \mathbf{x}[\mathcal{N}_i(p)], \quad (1)$$

where $\mathcal{N}_i(p)$ is the i -th position in the receptive field and $\mathbf{W}_i \in \mathbb{R}^{C_{out} \times C_{in}}$ is the corresponding weight slice. The output at each position is thus an accumulation of k^2 weighted contributions from the input. On the GPU, $\mathcal{G}_{out}$ is partitioned into N disjoint tiles $\mathcal{G}_{out} = T_1 \cup \dots \cup T_N$, $T_i \cap T_j = \emptyset$ for $i \neq j$, where each tile T_j is a contiguous spatial block of output positions. Tiles are processed in parallel; within each tile, positions are spatially adjacent, so their receptive fields overlap and lie in contiguous memory. This regularity enables cuDNN-class [17] throughput. We denote by $C \subseteq \mathcal{G}_{out}$ the set of positions actually computed by a given execution strategy. Dense convolution sets $C = \mathcal{G}_{out}$ unconditionally, which on embedded platforms often exceeds the latency budget.

B. Masked Convolution for Onboard Perception

1) *Mask Sources*: Many onboard vision systems exploit the fact that only a subset of the input changes between inference calls or is relevant to the current decision. Let $\mathcal{G}_{in} = \{1, \dots, H_{in}\} \times \{1, \dots, W_{in}\}$ denote the input spatial grid. These systems identify an input active mask $M_{in} \subseteq \mathcal{G}_{in}$ marking positions where the input has changed or is task-relevant. An output position p must be recomputed whenever its receptive field intersects the input mask, giving the output active mask:

$$M = \{p \in \mathcal{G}_{out} \mid \mathcal{N}(p) \cap M_{in} \neq \emptyset\}. \quad (2)$$

In a multi-layer network, the output mask of layer l becomes the input mask of layer $l+1$, propagating sparsity through the model. Three families of mask sources produce the initial M_{in} :

a) *Event-camera increments*.: Event cameras [18] are bio-inspired sensors in which each pixel independently fires when its brightness change exceeds a threshold, generating data only where intensity varies. They offer microsecond temporal resolution, high dynamic range, and negligible motion blur, making them widely used on UAVs and high-speed platforms. Events accumulated over a temporal window are differenced to reveal which positions received new activity, producing a naturally sparse mask [4], [5].

b) *Temporal frame differences*.: Standard frame-based cameras on mobile robots and autonomous vehicles capture video at a fixed rate. Between consecutive frames most of the scene remains static, especially under moderate ego-motion. Temporal differencing methods [6], [11] compare consecutive frames (or motion-compensated histories) and identify positions where pixel differences exceed a threshold; only outputs affected by these positions need recomputation.

c) *Learned spatial gating*.: Rather than relying on temporal change, dynamic networks [7] train a lightweight gating module that examines the current input and predicts, per spatial region, whether computation is needed for the downstream task. Regions classified as low-value are masked out, adapting the compute budget to task-relevant content per frame.

2) *Execution Paradigms*: Given a mask M , the system must compute $\mathbf{y}[p]$ only for $p \in M$. Existing methods follow one of two paradigms:

[TODO] Active ratio ρ (x-axis) vs. actual speedup over dense (y-axis) on Jetson Orin. Three mask sources, two paradigms. Dashed line = ideal linear scaling.

Fig. 1. Active ratio vs. actual convolution speedup on Jetson Orin for the three mask sources. The dashed line marks ideal (linear) scaling. Tile skipping and gather-scatter both fall far short.

a) *Tile skipping.*: A tile T_j is executed if and only if $T_j \cap M \neq \emptyset$; otherwise it is skipped entirely. The computed set is $C = \bigcup_{j: T_j \cap M \neq \emptyset} T_j$. Because non-skipped tiles run the unmodified dense pipeline, throughput is preserved. However, $|C|$ can be much larger than $|M|$ when active positions are scattered across many tiles: a single active position forces its entire tile to execute. SBNNet [8], Skip-Convolutions [9], Ev-Conv [4], and DeltaCNN [6] follow this paradigm.

b) *Gather-scatter.*: Active positions are extracted exactly: the system gathers $\{\mathbf{x}[\mathcal{N}(p)] : p \in M\}$ into a contiguous buffer, computes Eq. (1) for each, and scatters results back to the output grid. The computed set is $C = M$ exactly, so no computation is wasted. However, the gather, scatter, and irregular memory access patterns abandon the dense pipeline’s regular execution, reducing throughput to a fraction of tile-based methods. DGNet [7] and SAI [10] follow this paradigm.

C. The Gap: Sparsity Does Not Become Speedup

For a network with L convolution layers, we define the *active ratio* $\rho = \frac{1}{L} \sum_{l=1}^L |M^l|/|\mathcal{G}_{out}^l|$, the average fraction of output positions that require computation across all layers. We characterize masked-convolution efficiency along two axes: *throughput*, the rate of convolution operations per second (bounded by the dense pipeline), and *useful-compute ratio*, the fraction of executed operations that produce needed outputs ($|M|/|C|$). Tile skipping preserves throughput but sacrifices useful-compute ratio; gather-scatter achieves high useful-compute ratio but sacrifices throughput. Neither delivers both, so upstream sparsity does not translate into proportional latency reduction. Figure 1 quantifies this on Jetson Orin running XX on YY dataset: with ρ in the range XX–YY%, tile-skipping methods achieve only AA–BB% useful-compute ratio and gather-scatter methods reach only CC% of tile-skipping throughput, reducing convolution latency by only DD–EE% rather than the ideal ρ -linear scaling. This gap motivates WISEConv.

D. 3D Sparse Convolution: A Different Problem

3D sparse convolution (SpConv v2 [19], TorchSparse++ [20]) targets point-cloud workloads where input data is inherently irregular and the core challenge is imposing computational structure on scattered voxels. WISEConv faces a fundamentally different problem: the input resides on a regular dense grid with structured, stride-based access; the challenge is how to skip inactive outputs without leaving the dense pipeline’s already-efficient execution. Applying 3D sparse convolution directly to this 2D scenario performs poorly because it requires building coordinate hash tables and neighbor maps with mandatory GPU synchronization, and the per-frame mask changes prevent amortizing this cost across layers. On Jetson Orin, SpConv v2 applied to a 2D masked convolution with $\rho = \text{XX}\%$ runs $\text{XX}\times$ slower than the dense baseline due to construction overhead alone (Section ??).

III. WISECONV DESIGN

A. Overview

Onboard robotic perception demands real-time convolution under strict compute and power budgets; on embedded platforms, dense convolution often cannot meet the hard latency ceiling imposed by the control loop (Section II-A). Masked convolution exploits spatial sparsity to reduce convolution latency by computing only the outputs affected by active input regions. However, as shown in Section II-C, existing approaches fail to convert high sparsity into proportional latency reduction due to the tension between useful-compute ratio and throughput. WISEConv resolves this tension through a two-stage design (Figure 2):

- 1) **Construction stage.** A lightweight kernel scans the input active mask M_{in} and evaluates Eq. (2) on the fly, emitting a *worklist* $W = (w_1, \dots, w_{|M|})$, where each $w_j \in M \subseteq \mathcal{G}_{out}$ is the linear index of an active output position. Each thread block covers one tile T_j and compacts its active positions into a contiguous segment of W .
- 2) **Compute stage.** A convolution kernel consumes W as its coordinate source. For each position $w_j \in W$, the kernel decodes the spatial coordinate, reads the receptive-field input $\mathbf{x}[\mathcal{N}(w_j)]$, and accumulates Eq. (1) identically to the dense pipeline. The computed set is $C = \{w_1, \dots, w_{|M|}\} = M$, so no computation is wasted.

WISEConv achieves high useful-compute ratio because the worklist contains only positions that require computation ($C = M$, no wasted work). It achieves near-dense throughput through three mechanisms: the construction stage operates entirely in mask space with minimal overhead (Section III-B); the worklist preserves locality comparable to the dense traversal order (Section III-B); and the compute stage’s per-position datapath is identical to the dense pipeline (Section III-C).

B. Worklist Construction

The construction stage produces a worklist $W \in \mathbb{Z}^{|M|}$, a vector whose entries are exactly the positions in M .

[TODO] WISEConv overview. Left: input mask M_{in} on the spatial grid. Middle: construction stage scans tiles $T_1, \dots, T_N$, emits ordered worklist W . Right: compute stage consumes W , drives implicit GEMM, writes output y at active positions only. Show tile-respecting order in W .

Fig. 2. WISEConv two-stage pipeline. The construction stage scans the mask tile by tile and emits an ordered worklist. The compute stage drives the dense convolution datapath using only the worklist positions.

a) *Tiled scan.*: The kernel assigns one GPU thread per output position in $\mathcal{G}_{out}$, with each thread block covering one tile T_j . For each tile, the kernel determines its active subset:

$$M_j = \{p \in T_j \mid \mathcal{N}(p) \cap M_{in} \neq \emptyset\}, \quad (3)$$

which requires reading at most k^2 entries from the boolean mask M_{in} per position (no feature data accessed). The thread block sorts M_j in raster order to produce a sub-vector $s_j \in \mathbb{Z}^{|M_j|}$ and writes it into a contiguous segment of W (Algorithm 1).

b) *Worklist structure.*: The resulting worklist W is partitioned into N contiguous sub-vectors, one per tile. Let o_j denote the starting index of tile T_j 's segment:

$$W[o_j : o_j + |M_j|] = s_j, \quad j = 1, \dots, N. \quad (4)$$

Each s_j is internally ordered (raster order within T_j). The off-sets $\{o_1, \dots, o_N\}$ constitute a permutation of the prefix sums of $\{|M_1|, \dots, |M_N|\}$, determined by the runtime execution order of thread blocks. Define the *ideal worklist* W^* as the same sub-vectors arranged in tile-index order:

$$W^* = (s_1, s_2, \dots, s_N), \quad (5)$$

which corresponds to the subsequence of the dense raster traversal restricted to M . Both W and W^* consist of the same sub-vectors $\{s_j\}$; they differ only in the ordering of these sub-vectors.

c) *Locality analysis.*: The compute stage processes W in chunks of B_M consecutive entries. Let $n_{\text{tiles}}(W, b)$ denote the number of distinct tiles represented in chunk $W[b : b + B_M]$. Since each sub-vector s_j has length $|M_j|$, any chunk of B_M entries spans at least

$$n_{\text{tiles}}(W, b) \geq \lceil B_M / \max_j |M_j| \rceil \quad (6)$$

tiles, regardless of the sub-vector ordering. The same bound holds for W^* . Therefore the spatial spread of input data per chunk (and hence the cache working set) is equivalent for W and W^* ; the non-deterministic inter-tile ordering does not degrade locality.

Algorithm 1 Worklist Construction (per thread block)

Require: Tile T_j , input mask M_{in} , global counter cnt

Ensure: Worklist W , output mask M

```

1: for each position  $p \in T_j$  handled by this thread do
2:    $\text{active}(p) \leftarrow [\mathcal{N}(p) \cap M_{in} \neq \emptyset]$ 
3:    $M[p] \leftarrow \text{active}(p)$ 
4: end for
5:  $\text{ballot} \leftarrow \text{warp\_ballot}(\text{active}(p))$ 
6:  $n_w \leftarrow \text{popcount}(\text{ballot})$ 
7:  $\text{rank} \leftarrow \text{prefix\_popcount}(\text{ballot})$ 
8:  $\text{base} \leftarrow \text{atomicAdd}(\text{cnt}, n_w)$  (warp leader)
9: if  $\text{active}(p)$  then
10:   $W[\text{base} + \text{rank}] \leftarrow \text{linear\_index}(p)$ 
11: end if
```

d) *Cost analysis.*: The construction kernel launches $|\mathcal{G}_{out}|$ threads, each performing $O(k^2)$ boolean reads from M_{in} and at most one integer write to W . The total work is $O(|\mathcal{G}_{out}| \cdot k^2)$ in mask space. Because it touches only the single-bit mask (not the C_{in} -channel feature map), its runtime is a small fraction of the compute stage (measured at XX% in Section ??).

The construction stage also writes the output active mask M (one boolean per position), which serves as M_{in} for the next layer in a multi-layer network.

C. Worklist-driven Convolution

The compute stage evaluates Eq. (1) for exactly the positions in W . We cast this as an implicit GEMM whose M-dimension is indexed by the worklist rather than by a sequential scan of $\mathcal{G}_{out}$.

a) *GEMM formulation.*: Let $|W| = |M|$. The masked convolution computes:

$$\mathbf{Y}_{|M| \times C_{out}} = \mathbf{A}_{|M| \times k^2 C_{in}} \cdot \hat{\mathbf{W}}_{k^2 C_{in} \times C_{out}}, \quad (7)$$

where row j of $\mathbf{A}$ is the flattened receptive-field input for position w_j :

$$\mathbf{A}[j, :] = [\mathbf{x}[\mathcal{N}_1(w_j)]^\top, \dots, \mathbf{x}[\mathcal{N}_{k^2}(w_j)]^\top], \quad (8)$$

Algorithm 2 Worklist-driven Convolution (per tile)

Require: Worklist W , worklist length $|M|$, input $\mathbf{x}$, weight $\hat{\mathbf{W}}$

Ensure: Output $\mathbf{y}$ at positions in W

```

1: for each tile  $(b_m, b_n)$  in the GEMM grid in parallel do
2:   Initialize accumulator  $\mathbf{Y}_{\text{tile}} \leftarrow \mathbf{0}$ 
3:   for  $s = 1, \dots, \lceil k^2 C_{in} / B_K \rceil$  do
4:     Load input block: for each row  $j$  in tile  $b_m$ , decode
        $w_j$  via Eq. (9), read the corresponding slice of
        $\mathbf{x}[\mathcal{N}(w_j)]$ 
5:     Load weight block from  $\hat{\mathbf{W}}$ 
6:      $\mathbf{Y}_{\text{tile}} +=$  block multiply
7:   end for
8:   Write  $\mathbf{Y}_{\text{tile}}$  to  $\mathbf{y}$  at positions  $W[b_m : b_m + B_M]$ 
9: end for

```

and $\hat{\mathbf{W}}$ is the weight tensor $\mathbf{W}$ reshaped into a 2D matrix. The matrix $\mathbf{A}$ is never materialized; each element is computed on the fly from the worklist index w_j by decoding its spatial coordinate and reading the corresponding input position.

b) Coordinate decoding.: Given a linear index w_j , the kernel recovers the batch index n and spatial position (p_h, p_w) :

$$n = \lfloor w_j / (H_{out} W_{out}) \rfloor, \quad p_h = \lfloor (w_j \bmod H_{out} W_{out}) / W_{out} \rfloor, \quad (9)$$

The receptive-field input addresses follow from (p_h, p_w) and the layer's stride and padding parameters, identically to the dense pipeline.

c) Equivalence to dense execution.: For any single position w_j , the sequence of operations (decode coordinate, read $k^2 \times C_{in}$ input values, multiply-accumulate against $\mathbf{W}$, write C_{out} outputs) is identical to what the dense pipeline performs for that same position. WISEConv changes only *which* positions are processed (worklist instead of full grid) and *how many* ($|M|$ instead of $|\mathcal{G}_{out}|$). The arithmetic intensity, memory access pattern per position, and GPU occupancy model are preserved.

D. Multi-layer Integration

In a multi-layer CNN, WISEConv's construction stage produces both the worklist W and the output active mask M for each layer. The output mask directly serves as M_{in} for the next layer's construction stage, propagating sparsity through the network without additional mask-computation passes. WISEConv is a drop-in replacement for standard convolution layers: it accepts the same weight tensors and produces numerically identical outputs at active positions, requiring no model retraining or architectural changes.

ACKNOWLEDGMENTS

REFERENCES

[1] D. Wofk, F. Ma, T. Yang, S. Karaman, and V. Sze, "Fastdepth: Fast monocular depth estimation on embedded systems," in *International Conference on Robotics and Automation, ICRA 2019, Montreal, QC, Canada, May 20-24, 2019*. IEEE, 2019, pp. 6101–6108. [Online]. Available: <https://doi.org/10.1109/ICRA.2019.8794182>

[2] R. J. Bouwmeester, F. Paredes-Vallés, and G. C. H. E. de Croon, "Nanoflownet: Real-time dense optical flow on a nano quadcopter," in *IEEE International Conference on Robotics and Automation, ICRA 2023, London, UK, May 29 - June 2, 2023*. IEEE, 2023, pp. 1996–2003. [Online]. Available: <https://doi.org/10.1109/ICRA48891.2023.10161258>

[3] M. Sundermeyer, A. Mousavian, R. Triebel, and D. Fox, "Contact-grasnet: Efficient 6-dof grasp generation in cluttered scenes," in *IEEE International Conference on Robotics and Automation, ICRA 2021, Xi'an, China, May 30 - June 5, 2021*. IEEE, 2021, pp. 13 438–13 444. [Online]. Available: <https://doi.org/10.1109/ICRA48506.2021.9561877>

[4] S. Durvasula, Y. Guan, and N. Vijaykumar, "Ev-conv: Fast CNN inference on event camera inputs for high-speed robot perception," *IEEE Robotics Autom. Lett.*, vol. 8, no. 6, pp. 3174–3181, 2023. [Online]. Available: <https://doi.org/10.1109/LRA.2023.3259731>

[5] N. Messikommer, D. Gehrig, A. Loquercio, and D. Scaramuzza, "Event-based asynchronous sparse convolutional networks," in *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part VIII*, ser. Lecture Notes in Computer Science, A. Vedaldi, H. Bischof, T. Brox, and J. Frahm, Eds., vol. 12353. Springer, 2020, pp. 415–431. [Online]. Available: https://doi.org/10.1007/978-3-030-58598-3_25

[6] M. Parger, C. Tang, C. D. Twigg, C. Keskin, R. Wang, and M. Steinberger, "Deltacnn: End-to-end CNN inference of sparse frame differences in videos," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 2022, pp. 12 487–12 496. [Online]. Available: <https://doi.org/10.1109/CVPR52688.2022.01217>

[7] F. Li, G. Li, X. He, and J. Cheng, "Dynamic dual gating neural networks," in *2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021*. IEEE, 2021, pp. 5310–5319. [Online]. Available: <https://doi.org/10.1109/ICCV48922.2021.00528>

[8] M. Ren, A. Pokrovsky, B. Yang, and R. Urtasun, "Sbnet: Sparse blocks network for fast inference," in *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 2018, pp. 8711–8720. [Online]. Available: http://openaccess.thecvf.com/content_cvpr_2018/html/Ren/SBNet_Sparse_Blocks_CVPR_2018_paper.html

[9] A. Habibi, D. Abati, T. S. Cohen, and B. E. Bejnordi, "Skip-convolutions for efficient video processing," in *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*. Computer Vision Foundation / IEEE, 2021, pp. 2695–2704. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2021/html/Habibi/Skip-Convolution_for_Efficient_Video_Processing_CVPR_2021_paper.html

[10] Z. Xie, Z. Zhang, X. Zhu, G. Huang, and S. Lin, "Spatially adaptive inference with stochastic feature sampling and interpolation," in *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part I*, ser. Lecture Notes in Computer Science, A. Vedaldi, H. Bischof, T. Brox, and J. Frahm, Eds., vol. 12346. Springer, 2020, pp. 531–548. [Online]. Available: https://doi.org/10.1007/978-3-030-58452-8_31

[11] L. Cavigelli, P. Degen, and L. Benini, "Cbinfer: Change-based inference for convolutional neural networks on video data," in *Proceedings of the 11th International Conference on Distributed Smart Cameras, Stanford, CA, USA, September 5-7, 2017*, M. O. Arias-Estrada, C. Micheloni, H. K. Aghajan, O. I. Camps, and V. M. Brea, Eds. ACM, 2017, pp. 1–8. [Online]. Available: <https://doi.org/10.1145/3131885.3131906>

[12] Z. Teed and J. Deng, "RAFT: recurrent all-pairs field transforms for optical flow," in *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part II*, ser. Lecture Notes in Computer Science, A. Vedaldi, H. Bischof, T. Brox, and J. Frahm, Eds., vol. 12347. Springer, 2020, pp. 402–419. [Online]. Available: https://doi.org/10.1007/978-3-030-58536-5_24

[13] C. Wang, A. Bochkovskiy, and H. M. Liao, "Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*. IEEE, 2023, pp. 7464–7475. [Online]. Available: <https://doi.org/10.1109/CVPR52729.2023.00721>

[14] C. Yu, J. Wang, C. Peng, C. Gao, G. Yu, and N. Sang, "Bisenet: Bilateral segmentation network for real-time semantic segmentation," in *Computer Vision - ECCV 2018 - 15th European Conference, Munich, Germany, September 8-14, 2018, Proceedings, Part XIII*, ser. Lecture Notes in Computer Science, V. Ferrari, M. Hebert, C. Sminchisescu,

- and Y. Weiss, Eds., vol. 11217. Springer, 2018, pp. 334–349. [Online]. Available: https://doi.org/10.1007/978-3-030-01261-8_20
- [15] C. Wang, D. Xu, Y. Zhu, R. M. Martin, C. Lu, L. Fei-Fei, and S. Savarese, “Densefusion: 6d object pose estimation by iterative dense fusion,” in *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*. Computer Vision Foundation / IEEE, 2019, pp. 3343–3352. [Online]. Available: http://openaccess.thecvf.com/content_CVPR_2019/html/Wang_DenseFusion_6D_Object_Pose_Estimation_by_Iterative_Dense_Fusion_CVPR_2019_paper.html
- [16] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. [Online]. Available: <https://openreview.net/forum?id=YicbFdNTTy>
- [17] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, “cudnn: Efficient primitives for deep learning,” *CoRR*, vol. abs/1410.0759, 2014. [Online]. Available: <http://arxiv.org/abs/1410.0759>
- [18] G. Gallego, T. Delbrück, G. Orchard, C. Bartolozzi, B. Taba, A. Censi, S. Leutenegger, A. J. Davison, J. Conradt, K. Daniilidis, and D. Scaramuzza, “Event-based vision: A survey,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 1, pp. 154–180, 2022. [Online]. Available: <https://doi.org/10.1109/TPAMI.2020.3008413>
- [19] Y. Yan, Y. Mao, and B. Li, “SECOND: sparsely embedded convolutional detection,” *Sensors*, vol. 18, no. 10, p. 3337, 2018. [Online]. Available: <https://doi.org/10.3390/s18103337>
- [20] H. Tang, S. Yang, Z. Liu, K. Hong, Z. Yu, X. Li, G. Dai, Y. Wang, and S. Han, “Torchsparse++: Efficient training and inference framework for sparse convolution on gpus,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2023, Toronto, ON, Canada, 28 October 2023 - 1 November 2023*. ACM, 2023, pp. 225–239. [Online]. Available: <https://doi.org/10.1145/3613424.3614303>